

Código completo

```
import cv2
import numpy as np
import tensorflow as tf

print("TensorFlow version:", tf.__version__)

# =====
# ABRIR CÂMARA
# =====
cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)

if not cap.isOpened():
    print("Erro: não foi possível abrir a câmera")
    exit()

width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

print("Resolução:", width, height)

# =====
# CRIAR MODELO
# =====
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(height, width)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(25, activation="sigmoid", name="perceptron_1"),
    tf.keras.layers.Dense(25, activation="sigmoid", name="perceptron_2"),
    tf.keras.layers.Dense(2, activation="softmax", name="output")
])

print(model.summary())

# =====
# COMPILAR MODELO
# =====
model.compile(
    optimizer=tf.keras.optimizers.Adagrad(learning_rate=0.01),
```

```
loss="categorical_crossentropy",
metrics=["accuracy"]
)

# =====
# LISTAS DE TREINO
# =====
images = []
y = []

EPOCHS = 100

labels = ["Robots", "Lion TEA"]

print("\nCOMANDOS:")
print("1 -> Capturar classe Robots")
print("2 -> Capturar classe Lion TEA")
print("t -> Treinar modelo")
print("p -> Fazer previsão")
print("q ou ESC -> Sair\n")

# =====
# LOOP PRINCIPAL
# =====
while True:
    ret, frame = cap.read()

    if not ret:
        print("Erro ao capturar frame")
        break

    # Mostrar vídeo
    cv2.imshow("Video", frame)

    # Converter para grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Normalizar
    gray = gray / 255.0
```

```
# Ler tecla
k = cv2.waitKey(1) & 0xFF

grab_image = False

# =====
# SAIR
# =====
if k == 27 or k == ord("q"):
    break

# =====
# CLASSE 1
# =====
elif k == ord("1"):
    initial_class = np.array([1.0, 0.0])
    grab_image = True

# =====
# CLASSE 2
# =====
elif k == ord("2"):
    initial_class = np.array([0.0, 1.0])
    grab_image = True

# =====
# TREINO
# =====
elif k == ord("t"):
    if len(images) == 0:
        print("Nenhuma imagem capturada para treino.")
    else:
        print("A treinar modelo...")
        model.fit(
            np.array(images),
            np.array(y),
            epochs=EPOCHS,
            verbose=1
        )
        print("Treino concluído.")
```

```
# =====
# PREVISÃO
# =====
elif k == ord("p"):
    if len(images) == 0:
        print("Modelo ainda não treinado.")
    else:
        predict = model.predict(np.array([gray]), verbose=0)

        predicted_index = np.argmax(predict)
        predicted_label = labels[predicted_index]

        print("Classe prevista:", predicted_label)
        print("Probabilidades:", predict)

# =====
# GUARDAR IMAGEM
# =====
if grab_image:
    images.append(gray)
    y.append(initial_class)

    print("Imagem capturada para:", labels[np.argmax(initial_class)])
    print("Total imagens:", len(images))

# =====
# FECHAR
# =====
cap.release()
cv2.destroyAllWindows()
```

Como usar

Capturar imagens

Pressiona:

- **1** → Robots
- **2** → Lion TEA

Cada clique guarda uma imagem para treino.

Exemplo:

- mostra objeto Robots → tecla **1**
- mostra pacote chá → tecla **2**

Captura várias imagens.

Sugestão:

- 20 a 50 por classe
-

Treinar

Pressiona:

t

Fazer previsão

Pressiona:

p

Sair

q

ou

ESC

EXPLICAÇÃO LINHA POR LINHA

Importa a biblioteca OpenCV para captura de vídeo e processamento de imagem

`import cv2`

Importa a biblioteca NumPy para trabalhar com arrays e operações matemáticas

`import numpy as np`

Importa o TensorFlow para criação e treino da rede neuronal

`import tensorflow as tf`

Mostra no ecrã a versão instalada do TensorFlow

`print("TensorFlow version:", tf.__version__)`

=====

ABRIR A CÂMARA

=====

Abre a webcam com índice 0 usando o backend DirectShow (Windows)

`cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)`

Verifica se a câmara foi aberta com sucesso

`if not cap.isOpened():`

Mostra mensagem de erro caso não consiga abrir

`print("Erro: não foi possível abrir a câmara")`

Termina a execução do programa

`exit()`

```
# Obtém a largura da imagem capturada pela câmara
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))

# Obtém a altura da imagem capturada pela câmara
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

# Mostra a resolução do vídeo capturado
print("Resolução:", width, height)

# =====

# CRIAR O MODELO DE REDE NEURONAL

# =====

# Cria um modelo sequencial do Keras
model = tf.keras.Sequential([

    # Define a forma de entrada da imagem (altura e largura)
    tf.keras.layers.Input(shape=(height, width)),

    # Converte a matriz da imagem num vetor unidimensional
    tf.keras.layers.Flatten(),

    # Primeira camada oculta com 25 neurónios
    tf.keras.layers.Dense(25, activation="sigmoid", name="perceptron_1"),

    # Segunda camada oculta com 25 neurónios
    tf.keras.layers.Dense(25, activation="sigmoid", name="perceptron_2"),
```

```
# Camada de saída com 2 classes
tf.keras.layers.Dense(2, activation="softmax", name="output")
])

# Mostra o resumo da arquitetura da rede
print(model.summary())

# =====

# COMPILAR O MODELO

# =====

# Configura o modelo para treino
model.compile(

# Define o algoritmo de otimização
optimizer=tf.keras.optimizers.Adagrad(learning_rate=0.01),

# Define a função de perda para classificação
loss="categorical_crossentropy",

# Define a métrica de avaliação
metrics=["accuracy"]
)

# =====

# LISTAS PARA TREINO
```



```
# =====

# Lista onde serão guardadas as imagens

images = []

# Lista onde serão guardadas as classes

y = []

# Número de épocas de treino

EPOCHS = 100

# Lista com os nomes das classes

labels = ["Robots", "Lion TEA"]

# Mostra os comandos disponíveis

print("\nCOMANDOS:")

print("1 -> Capturar classe Robots")

print("2 -> Capturar classe Lion TEA")

print("t -> Treinar modelo")

print("p -> Fazer previsão")

print("q ou ESC -> Sair\n")

# =====

# CICLO PRINCIPAL

# =====

# Inicia um ciclo infinito
```

while True:

 # Captura um frame da webcam

 ret, frame = cap.read()

 # Verifica se a captura foi feita com sucesso

 if not ret:

 print("Erro ao capturar frame")

 break

 # Mostra o vídeo em tempo real

 cv2.imshow("Video", frame)

 # Converte a imagem de BGR para tons de cinzento

 gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

 # Normaliza os valores dos píxeis entre 0 e 1

 gray = gray / 255.0

 # Lê a tecla premida pelo utilizador

 k = cv2.waitKey(1) & 0xFF

 # Inicializa a variável de controlo da captura

 grab_image = False

 # =====

 # SAIR DO PROGRAMA

```
# =====

# Se premir ESC ou q
if k == 27 or k == ord("q"):
    break

# =====

# CLASSE 1

# =====

# Se premir a tecla 1
elif k == ord("1"):

    # Define a classe como Robots
    initial_class = np.array([1.0, 0.0])

    # Activa a captura da imagem
    grab_image = True

# =====

# CLASSE 2

# =====

# Se premir a tecla 2
elif k == ord("2"):

    # Define a classe como Lion TEA
```

```
initial_class = np.array([0.0, 1.0])

# Activa a captura da imagem
grab_image = True

# =====
# TREINAR A REDE
# =====

# Se premir a tecla t
elif k == ord("t"):

    # Verifica se existem imagens capturadas
    if len(images) == 0:
        print("Nenhuma imagem capturada para treino.")
    else:
        # Mostra mensagem de início do treino
        print("A treinar modelo...")

        # Treina a rede neuronal
        model.fit(
            np.array(images), # imagens de treino
            np.array(y),      # classes correspondentes
            epochs=EPOCHS,    # número de épocas
            verbose=1         # mostra progresso
        )
```

```
# Mostra mensagem de conclusão
print("Treino concluído.")

# =====

# PREVISÃO

# =====

# Se premir a tecla p
elif k == ord("p"):

    # Verifica se já existem dados de treino
    if len(images) == 0:
        print("Modelo ainda não treinado.")
    else:
        # Faz a previsão da classe da imagem actual
        predict = model.predict(np.array([gray]), verbose=0)

        # Obtém o índice da classe com maior probabilidade
        predicted_index = np.argmax(predict)

        # Obtém o nome da classe prevista
        predicted_label = labels[predicted_index]

        # Mostra a classe prevista
        print("Classe prevista:", predicted_label)

        # Mostra as probabilidades
```

```
print("Probabilidades:", predict)

# =====

# GUARDAR IMAGEM

# =====

# Se a captura estiver activa
if grab_image:

    # Guarda a imagem
    images.append(gray)

    # Guarda a classe correspondente
    y.append(initial_class)

    # Mostra mensagem de confirmação
    print("Imagem capturada para:", labels[np.argmax(initial_class)])

    # Mostra total de imagens guardadas
    print("Total imagens:", len(images))

# =====

# FECHAR PROGRAMA

# =====

# Liberta a webcam
cap.release()
```

Fecha todas as janelas abertas pelo OpenCV

`cv2.destroyAllWindows()`